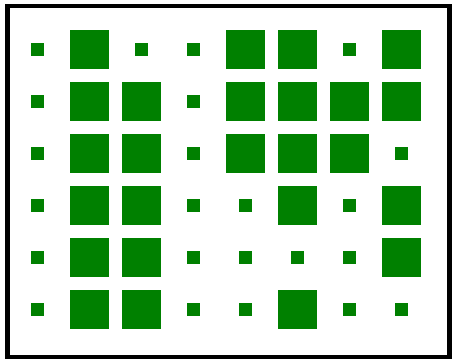
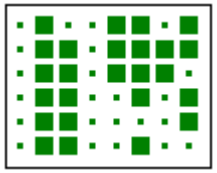


Class 6
C Introduction

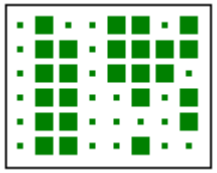
CS 330
Programming Languages and Compilers

Language Discussion Assumption



- Given that CS-209 and CS-226 are prerequisites for this class, I am assuming each of you has familiarity with basic programming concepts like variables, assignment statements, loops, and functions
- Further, I assume each of you has a working understanding of Java and Python, in that you can express the concepts listed above in each and are able to write simple programs without my reviewing the constructs

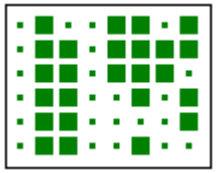
C Background



C is a general-purpose programming language which features **economy of expression**, modern control flow and data structures, and a rich set of operators. C is not a “very high level” language, not a “big” one, and is not specialized to an particular area of application. But its **absence of restrictions** and its generality make it more convenient and effective for many tasks than supposedly more powerful languages

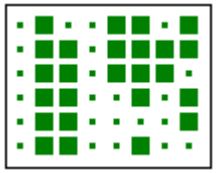
The C Programming Language; Kernighan, Brian and Ritchie, Dennis; 1978

C Language Popularity 1/2



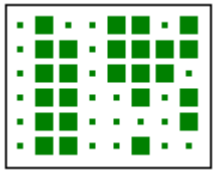
- Since its debut in the mid 1970's, C has been a popular language for implementing compute-intensive software
  - Operating systems
    - Initial use was to create the UNIX operating system and utility programs (grep, awk, sed…)
  - Database management systems
  - Audio, video and image editing tools

C Language Popularity 2/2



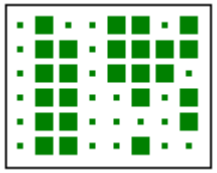
- Why such popularity?
  - It bridges a gap between very low-level (tedious and error prone) and very high-level (slow and functionally limited) languages
  - Typical C compiler outputs assembly language – so what?

C Language Key Differentiators 1/2



- C is fairly “low level” with regards to data types
  - Natural information that computers use (characters, numbers and addresses)
- No operations to deal directly with composite objects such as character strings, sets, or arrays
- Single-threaded control flow. No parallel operation or synchronization primitives

C Language Key Differentiators 2/2

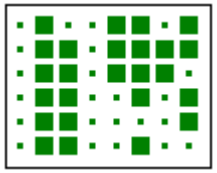


- No storage allocation system managed by the language other than static definitions and a stack.
  - No concept of managed heap or garbage collection



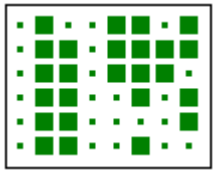
- No built-in I/O. All reading and writing of information is done through functions

C Language Key Design Concepts 1/2



- Fundamental data types are characters, integers (different sizes) and floating point numbers
- Derived data types using pointers, arrays, structures, unions, and functions
- Memory address arithmetic support
- Arguments to functions are passed by value (e.g. the value is copied to a separate variable in the function)

C Language Key Design Concepts 2/2

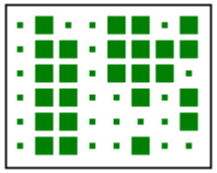


- Functions may be called recursively with argument and local variables independently stored for each invocation
- Passive conversion is done if assignment types disagree



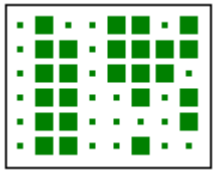
- Visible characters such as braces and semicolons indicate blocks and statements
- Whitespace is largely irrelevant. Use indentation to highlight logical structure

An Aside on Code Formatting

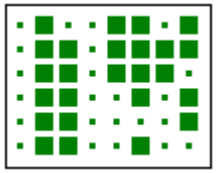


- Since C doesn't require a specific structure for the source code, there may be wide variance in the appearance of code from programmer to programmer
  - Most programming shops set standards to prevent significant formatting differences
    - What issues might arise from vastly different layouts of code?

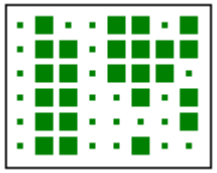
From the K&R Book



We will always indent statements within a block so you can see what statements are inside the block. The ***indentation*** *emphasizes the* ***logical structure*** of the program. Although C compilers do not care about how a program looks, *proper indentation and spacing are critical to making programs* ***easy for people to read***. We recommend writing only one statement per line and using blanks around operators. The position of braces is less important but people hold passionate beliefs. Pick a style and use it consistently.



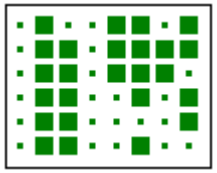
C Initial Tutorial



Key Set Up

https://bits.monead.com/cs330

Set Up Git Project for Individual Work



- Open a terminal and change directory into your home directory:

```
cd ~
```

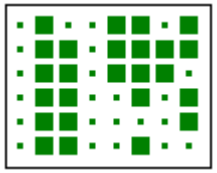
- Clone the individual lab work project:

```
git clone USER_NAME@bits.monead.com:cs330-IndividualLabWork
```

  - USER_NAME is your Linux account username

- You will be using this git project to submit all your individual lab work for this class
- You will not clone it again, unless you want to make it available on another computer

Obtain the Lab 1 Starting Code

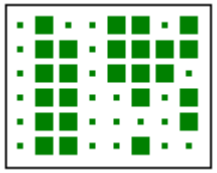


- Open a terminal and change directory into your **cs330-IndividualLabWork** directory
- Use git to obtain the lab 1 code:

```
git pull
```

- The directory **lab1-C-Introduction** should be added to your project
  - It contains several example C programs which we will explore
- You will be writing additional C programs for your lab assignment this week

Hello, World



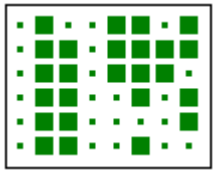
- Following tradition, we start with the ubiquitous **“hello, world”** program

  File **`hello.c`**

```
#include <stdio.h>
int main() {
  printf("hello, world\n");
}
```

- Compile it with the command: **`cc hello.c`**
- Assuming no syntax errors, a file called **`a.out`** will be created. Run the program by typing **`./a.out`**

Exploring Hello, World



```
#include <stdio.h>


int main() {


  printf("hello, world\n");

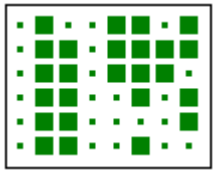

}
```

Preprocessor instruction to insert content from a separate file named **`stdio.h`**, a standard library file

Define a function named **`main`** that declares no parameters and returns an integer. Blocks are enclosed in braces. All C programs start at main

Call the library function **`printf`** to output a sequence of characters to the standard output; **`\n`** is a character representing a newline. This is one program statement. Statements are terminated with a semicolon

Equivalent Hello, World



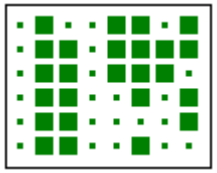
- The **`printf`** function does not insert a new line automatically

  File: **`hello2.c`**

```
#include <stdio.h>
   int main() {
            printf("hello, ");
            printf("world");
            printf("\n");
   }
```

- Compile: **`cc hello2.c`**
- Run the program: **`./a.out`**

Variables and Expressions



- A program to convert Fahrenheit to Celsius

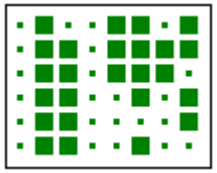
  TempC = (5/9)(TempF – 32)

- File: **`fah2cel.c`**

```
#include <stdio.h>
/* print Fahrenheit-Celsius table
   for Fahrenheit temps 0, 20, ..., 300 */
int main() {
        int fahr, celsius;
        int lower, upper, step;
        lower = 0;
        upper = 300;
        step = 20;
        fahr = lower;
        while (fahr <= upper) {
                celsius = 5 * (fahr - 32) / 9;
                printf("%d\t%d\n", fahr, celsius);
                fahr = fahr + step;
        }

}
```

Comments



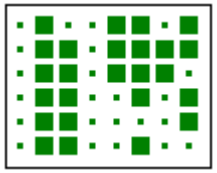
```
/* print Fahrenheit-Celsius table
      for Fahrenheit temps 0, 20, ..., 300 */
```

- Comments are formatted just like Java's single and multi-line forms

  single line comment: `//`

  multi-line comment: `/* … */`

Declaration

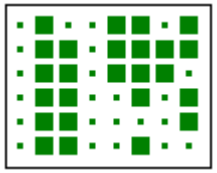


```
int fahr, celsius;
int lower, upper, step;
```

- Variables must be declared before use
- Declaration determines data type
- More than one variable may be included in declaration statement
- Values in declared variables are undefined; **they are not initialized**
  - Very different from Java and Python

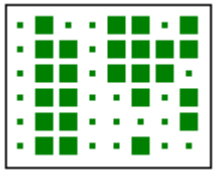


C's Data Types



- C provides several basic data types
  - `int` – integer
  - `float` – floating point
  - `char` – character
  - `short` – short integer
  - `long` – long integer
  - `double` – double-precision floating point
- Sizes of types are machine dependent; different from Java and Python
- Typically **`int`** is defined as the natural size (word size) for the system
- *No string type*

Assignments and Initialization



```
lower = 0;
upper = 300;
step = 20;
fahr = lower;
```

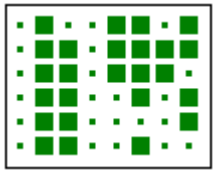
- Assign a value to a variable
- Type of the value and variable should agree
- C will coerce incorrect value type to fit in a variable; source of bugs



- Declaration and initialization may be combined in a single statement

```
int lower = 0;
```

Statement: `while`



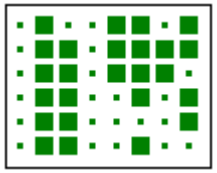
```
while (fahr <= upper)
    statement;
```

- Value in parentheses is evaluated

  0 (zero) is false and non-0 is true

- There is no Boolean type in C.
  Relational and logical operators result in 0 or 1
- The body of a **`while`** is a single statement;
  blocks are used to group more than one statement
- Same relational and logical operators as Java

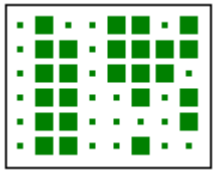
Expressions



```
celsius = 5 * (fahr - 32) / 9;
printf("%d\t%d\n", fahr, celsius);
fahr = fahr + step;
```

- P(MD)(AS) rules apply in C
- Integer division truncates fractional result
- Operators similar to Java (not overloaded +)
- `printf` function has a feature for displaying variable values using conversion specifications which start with a percent sign
- `\t` is the tab character

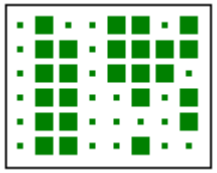
`printf` Conversion Specifications



```
int printf(char* format, arg1, arg2, ...)
```

- `format` is an array of characters
  - May include conversion specifications which begin with a % (next slide)
- Between % and conversion character may be
  - A minus sign which specifies left alignment
  - A number that specifies the minimum field width, padding will be added as necessary
  - A period followed by a precision

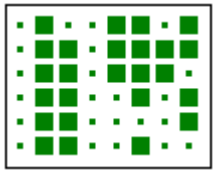
`printf` Conversion Characters



Character	Argument type; Printed As
`d, i`	`int`; decimal number
`o`	`int`; unsigned octal number (without a leading zero)
`x, X`	`int`; unsigned hexadecimal number (without a leading `0x` or `0X`), using `abcdef` or `ABCDEF` for 10, ...,15.
`u`	`int`; unsigned decimal number
`c`	`int`; single character
`s`	`char *`; print characters from the string until a `'\0'` or the number of characters given by the precision.
`f`	`double`; `[-]`*m.dddddd*, where the number of *d*'s is given by the precision (default 6).
`e, E`	`double`; `[-]`*m.dddddd*`e+/-`*xx* or `[-]`*m.dddddd*`E+/-`*xx*, where the number of *d*'s is given by the precision (default 6).
`g, G`	`double`; use `%e` or `%E` if the exponent is less than -4 or greater than or equal to the precision; otherwise use `%f`. Trailing zeros and a trailing decimal point are not printed.
`p`	`void *`; pointer (implementation-dependent representation).

To print a percent sign, use two: `printf("%%");`

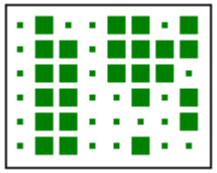
Using Floating Point Values



- Use floating point to avoid integer truncation
- File: `fah2cel2.c`

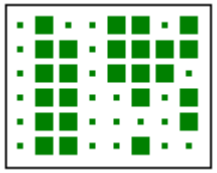
```
#include <stdio.h>
/* print Fahrenheit-Celsius table
   for Fahrenheit temps 0, 20, ..., 300 */
int main() {
        float fahr, celsius;
        int lower, upper, step;
        lower = 0;
        upper = 300;
        step = 20;
        fahr = lower;
        while (fahr <= upper) {
                celsius = 5.0 / 9.0 * (fahr - 32);
                printf("%3.0f\t%6.1f\n", fahr, celsius);
                fahr = fahr + step;
        }
}
```

What's different with this version?



- Look through the differences and discuss what is happening
- For example consider
  - The `fahr=lower;` assignment
  - The `%3.0f` and `%6.1f` conversions in the `printf`
    - What if we left those as `%d`?
- Other questions before we move on?

Statement: `for`

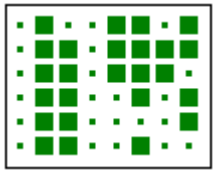


- Another variant of Fahrenheit to Celsius
- File: `fah2cel3.c`

```
#include <stdio.h>
// print Fahrenheit-Celsius table
int main() {
        int fahr;
        for (fahr = 0;fahr <= 300;fahr = fahr + 20)
                printf("%3d %6.1f\n", fahr,
                        (5.0 / 9.0) * (fahr - 32));
}
```

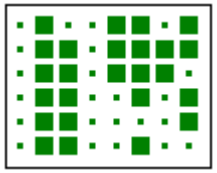
- **`for`** loop formatted like Java's standard **`for`** loop
- What else is going on in this program?
  - Note: we are using only one variable

Control Flow: `if`, `else` and `do/while`



- The **`if`** and **`else`** keywords work in C just like Java
  - The condition in the `if` test also uses a numeric type where 0 (zero) is false and all other values are true
- C also supports a **`do/while`** loop construct, again with syntax matching that of Java
- There is no C equivalent for Java's enhanced `for` (for each) loop

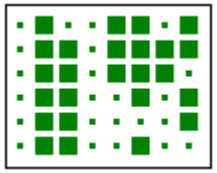
Symbolic Constants



- Fourth variant, avoiding “magic numbers”
- File: `fah2cel4.c`

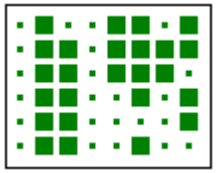
```
#include <stdio.h>
#define LOWER 0         // lower limit of table
#define UPPER 300       // upper limit of table
#define STEP 20         // step size for table
// print Fahrenheit-Celsius table
int main() {
    int fahr;
    for (fahr = LOWER;fahr <= UPPER;fahr = fahr + STEP)
        printf("%3d %6.1f\n", fahr, (5.0 / 9.0) * (fahr - 32));
}
```

- **`#define`** associates one character pattern with another. These are not variables and the `#define` line is not a statement, it is a preprocessor directive **defining a macro**



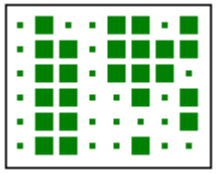
C: Basic I/O

Character Streams



- The standard I/O operations in C work with streams of characters
  - Regardless of the data source these methods process it as characters
- Remember that text files often end each line with a newline character (\n)
  - That convention is handy for using text files to store simple collections of data

Character Streams: Input (Keyboard)

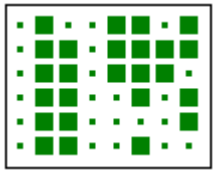


- **`getchar()`** is the basic function to read a character from standard input
  - Returns the next available character or -1 if the end of the file has been reached

    ```
    c = getchar()
    ```

  - Puts the next character value in the variable `c`
- The value returned is an **`int`**
  - The `int` is the character value (e.g. A is 65 when using ISO standard encoding)

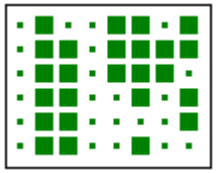
ASCII Table



https://commons.wikimedia.org/wiki/File:ASCII-Table.svg

Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]
1	1	1	1	[START OF HEADING]
2	2	10	2	[START OF TEXT]
3	3	11	3	[END OF TEXT]
4	4	100	4	[END OF TRANSMISSION]
5	5	101	5	[ENQUIRY]
6	6	110	6	[ACKNOWLEDGE]
7	7	111	7	[BELL]
8	8	1000	10	[BACKSPACE]
9	9	1001	11	[HORIZONTAL TAB]
10	A	1010	12	[LINE FEED]
11	B	1011	13	[VERTICAL TAB]
12	C	1100	14	[FORM FEED]
13	D	1101	15	[CARRIAGE RETURN]
14	E	1110	16	[SHIFT OUT]
15	F	1111	17	[SHIFT IN]
16	10	10000	20	[DATA LINK ESCAPE]
17	11	10001	21	[DEVICE CONTROL 1]
18	12	10010	22	[DEVICE CONTROL 2]
19	13	10011	23	[DEVICE CONTROL 3]
20	14	10100	24	[DEVICE CONTROL 4]
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]
22	16	10110	26	[SYNCHRONOUS IDLE]
23	17	10111	27	[ENG OF TRANS. BLOCK]
24	18	11000	30	[CANCEL]
25	19	11001	31	[END OF MEDIUM]
26	1A	11010	32	[SUBSTITUTE]
27	1B	11011	33	[ESCAPE]
28	1C	11100	34	[FILE SEPARATOR]
29	1D	11101	35	[GROUP SEPARATOR]
30	1E	11110	36	[RECORD SEPARATOR]
31	1F	11111	37	[UNIT SEPARATOR]
32	20	100000	40	[SPACE]
33	21	100001	41	!
34	22	100010	42	"
35	23	100011	43	#
36	24	100100	44	$
37	25	100101	45	%
38	26	100110	46	&
39	27	100111	47	'
40	28	101000	50	(
41	29	101001	51	)
42	2A	101010	52	*
43	2B	101011	53	+
44	2C	101100	54	,
45	2D	101101	55	-
46	2E	101110	56	.
47	2F	101111	57	/
48	30	110000	60	0
49	31	110001	61	1
50	32	110010	62	2
51	33	110011	63	3
52	34	110100	64	4
53	35	110101	65	5
54	36	110110	66	6
55	37	110111	67	7
56	38	111000	70	8
57	39	111001	71	9
58	3A	111010	72	:
59	3B	111011	73	;
60	3C	111100	74	<
61	3D	111101	75	=
62	3E	111110	76	>
63	3F	111111	77	?
64	40	1000000	100	@
65	41	1000001	101	A
66	42	1000010	102	B
67	43	1000011	103	C
68	44	1000100	104	D
69	45	1000101	105	E
70	46	1000110	106	F
71	47	1000111	107	G
72	48	1001000	110	H
73	49	1001001	111	I
74	4A	1001010	112	J
75	4B	1001011	113	K
76	4C	1001100	114	L
77	4D	1001101	115	M
78	4E	1001110	116	N
79	4F	1001111	117	O
80	50	1010000	120	P
81	51	1010001	121	Q
82	52	1010010	122	R
83	53	1010011	123	S
84	54	1010100	124	T
85	55	1010101	125	U
86	56	1010110	126	V
87	57	1010111	127	W
88	58	1011000	130	X
89	59	1011001	131	Y
90	5A	1011010	132	Z
91	5B	1011011	133	[
92	5C	1011100	134	\
93	5D	1011101	135	]
94	5E	1011110	136	^
95	5F	1011111	137	_
96	60	1100000	140	`
97	61	1100001	141	a
98	62	1100010	142	b
99	63	1100011	143	c
100	64	1100100	144	d
101	65	1100101	145	e
102	66	1100110	146	f
103	67	1100111	147	g
104	68	1101000	150	h
105	69	1101001	151	i
106	6A	1101010	152	j
107	6B	1101011	153	k
108	6C	1101100	154	l
109	6D	1101101	155	m
110	6E	1101110	156	n
111	6F	1101111	157	o
112	70	1110000	160	p
113	71	1110001	161	q
114	72	1110010	162	r
115	73	1110011	163	s
116	74	1110100	164	t
117	75	1110101	165	u
118	76	1110110	166	v
119	77	1110111	167	w
120	78	1111000	170	x
121	79	1111001	171	y
122	7A	1111010	172	z
123	7B	1111011	173	{
124	7C	1111100	174	|
125	7D	1111101	175	}
126	7E	1111110	176	~
127	7F	1111111	177	[DEL]

Character Streams: Output (Terminal)

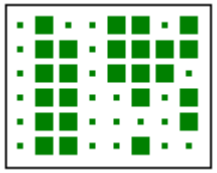


- **`putchar(c)`** is used to write one character to the standard output

  ```
  putchar(c)
  ```

  - Writes the value in variable **`c`** to the standard output
- Remember, the output value is an `int` and will be interpreted as a character using the current character set of the environment

Copy Input to Output



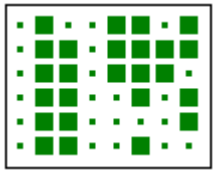
- Use `getchar()` and `putchar(c)` to copy from the standard input to the standard output
- Read and write characters until EOF is matched (where is macro EOF defined?)

File: **`filecopy.c`**

```
#include <stdio.h>
/* copy input to output, version 1 */
int main() {
        int c;
        c = getchar();
        while (c != EOF) {
                putchar(c);
                c = getchar();
        }
}
```

- How do we use this program?

Copy Input to Output, Version 2



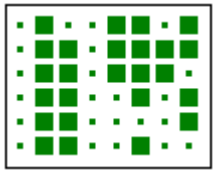
- Streamline the code, removing the redundant call to `getchar()`

File: **`filecopy2.c`**

```
#include <stdio.h>
/* copy input to output, version 2 */
int main() {
        int c;
        while ((c = getchar()) != EOF)
                putchar(c);
}
```

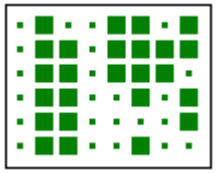
- Is this easier or harder to write, read, and maintain versus the previous version?

Counting Chars, Words and Lines



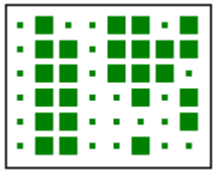
- Calculate some metadata about the input
  - File: **`filecount.c`**

```
#include <stdio.h>
#define IN 1    // Inside a word
#define OUT 0   // Outside a word
/* count lines, words and characters in input */
int main() {
        int c, nl, nw, nc, state;
        state = OUT;
        nl = nw = nc = 0;
        while ((c = getchar()) != EOF) {
                nc++;
                if (c == '\n')
                        nl++;
                if (c == ' ' || c == '\n' || c == '\t')
                        state = OUT;
                else if (state == OUT) {
                        state = IN;
                        nw++;
                }
        }
        printf("%d %d %d\n", nl, nw, nc);
}
```



C: Arrays

Array Basics



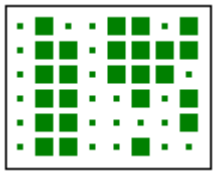
- C arrays are fixed size, set when created
- Array elements are expected to be one type
- Array indexes start at 0
- Arrays are declared using square brackets

```
int ndigit[10];
```

Declares and allocates, on the stack, a 10 element array of integers

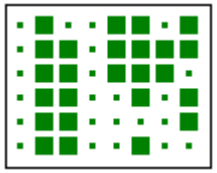
C: No Array Bounds Checking





- Whatever index your program asks for, C will attempt to access that location
  - Value in that memory location is returned
- Can cause a variety of bugs including the program appearing to work for a while, unrelated variables being corrupted, the program crashing…
- No IndexOutOfBounds exception or automatically zeroed-out/null values
- Example of C favoring speed over safety

Digit Counter (1/2)



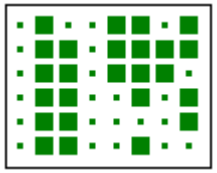
- Program to count frequency of each digit found in the input, using array to hold counts
- File: **`digitcounter.c`** (code is across 2 slides)

```
#include <stdio.h>

int main() {
    int c, i, nwhite, nother;
    int ndigit[10];

    nwhite = nother = 0;
    for (i = 0;i < 10;++i)
        ndigit[i] = 0;
```

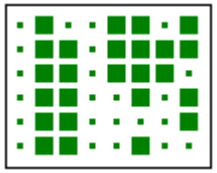
Digit Counter (2/2)



```
    while ((c = getchar()) != EOF)
        if (c >= '0' && c <= '9')
            ++ndigit[c - '0'];
        else if (c == ' ' || c == '\n' || c == '\t')
            ++nwhite;
        else
            ++nother;

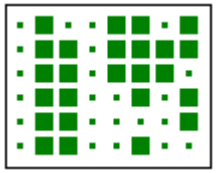

    printf("digits = ");
    for (i = 0;i < 10;++i)
        printf(" %d", ndigit[i]);
    printf(", white space = %d, other = %d\n", nwhite, nother);
}
```

- What are some interesting things you see being done in the code?



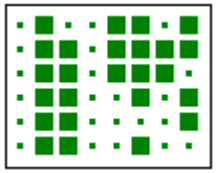
C: Functions

Function Basics 1/2



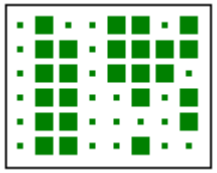
- A function provides a way to encapsulate and name some statements, which can be used (called) from multiple points in the program
- Breaking a program into small, single purpose, functions makes it easier to understand, test, and maintain

Function Basics 2/2



- Like Python and Java functions and methods, a C function may accept parameters and return a value
- Arguments are passed by value – the value sent by the caller is copied into the function's parameter
- Each call to a function is independent, values of parameters and (non-static) local variables are lost between calls (stateless)
  - Supports recursion

Calculate Exponentiation



- Function to calculate exponential values

  File: **`calcpower.c`**

```
#include <stdio.h>
/* declare the power function */
int power(int m, int n);
/* test the power function */
int main() {
        int i;
        for (i = 0;i < 10;++i)
                printf("%d %d %d\n", i, power(2, i), power(3, i));
}
/* power: raise base to n-th power; n >= 0 */
int power(int base, int n) {
        int i, p;
        p = 1;
        for (i = 1;i <= n;++i)
                p = p * base;
        return p;
}
```

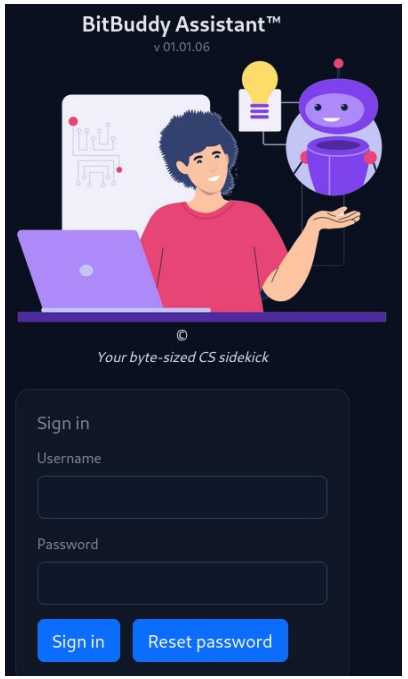
- Why declare the function at the top of the code?

Wednesday's Class

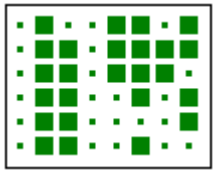
- You will start lab 1 (C programming) on Wednesday
- You will also be given your credentials to access BitBuddy, our LLM tool
- The lab will be due the following Monday night (9/28)

Lab 1
C Introduction

CS 330
Programming Languages and Compilers



Homework



- Friday (9/25, before class):
  - *Optional*: skim introductory sections of **regular expression document**
  - LLM Reading: 9 habits… (Perusall annotations)



- Sunday (9/27):
  - Journal entry for week 3
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)
- Monday (9/28):
  - Lab 1 due (C coding, team and individual)
  - Remember to include LLM usage choices for exercises in the README files